GNN-Based Object Recognition Through SLIC Superpixel Graphs on CIFAR-10

A Detailed Research Paper Based on the Supplied Project Notebook and Draft Report

Author: Harmit Patel
Department of Computer Science, University of Windsor
Windsor, Ontario, Canada

Abstract

Object recognition is an important component of robotic perception because a robot must identify the objects around it before it can plan actions such as grasping, sorting, navigation, or interaction. Most modern image classification systems use convolutional neural networks, which process images as fixed rectangular pixel grids. This project investigates a different representation: each image is first divided into perceptually meaningful superpixels using the Simple Linear Iterative Clustering (SLIC) algorithm, and the resulting regions are converted into a graph. In this graph, each superpixel becomes a node, spatially adjacent regions become edges, and each node is described by mean colour and normalized location features. Three graph neural network architectures are then compared on CIFAR-10: a GraphSAGE-based baseline GNN, a Graph Convolutional Network (GCN), and a Graph Attention Network (GAT). Using 2,000 training images and 500 test images, the GCN achieves the strongest result with 29.40% test accuracy, outperforming GAT at 25.20% and the GraphSAGE baseline at 20.60%. The results show that graph-based image representations can capture useful spatial structure, but they also show clear limitations when only simple colour-position node features are used. The paper concludes that superpixel GNNs are promising for compact visual reasoning, especially in robotics, but require richer visual features and larger training sets to become competitive with stronger vision baselines.

Keywords: graph neural networks, superpixels, SLIC, CIFAR-10, GraphSAGE, GCN, GAT, robotic perception, object recognition.

1. Introduction

Autonomous robots need reliable perception systems. A household robot, for example, must distinguish between a cup, a shoe, a toy, a pet, and furniture before it can choose an appropriate action. Object recognition is therefore not just a classification problem; it is a foundation for physical decision-making. If the perception module fails, the robot may select the wrong object, plan the wrong motion, or misunderstand the scene.

Standard computer vision models usually process images as regular grids of pixels. This is natural for convolutional neural networks because convolutional filters slide over local rectangular neighborhoods. However, real objects are not always aligned with rectangular grids. Boundaries may be curved, regions may have irregular shapes, and meaningful parts of an object may occupy uneven areas in the image. A graph representation can model these irregular relationships more directly because graph nodes and edges do not need to follow a fixed rectangular layout.

This project explores a graph-based image classification pipeline. First, each CIFAR-10 image is resized and segmented into superpixels using SLIC. A superpixel is a group of nearby pixels that are visually similar. Instead of using every pixel as a separate unit, the method uses each superpixel as a compact region. Each region becomes a graph node, and two nodes are connected if the corresponding superpixels share a boundary. This converts an image into a graph that can be processed by graph neural networks.

The central research question is: can superpixel graph representations support object recognition on CIFAR-10, and which GNN architecture performs best under this representation? To answer this question, the project compares three architectures: a GraphSAGE-based baseline model, a GCN model, and a GAT model. The comparison uses the same graph construction method and similar training conditions for all models.

2. Research Gap and Motivation

The main research gap addressed by this project is the limited practical comparison of basic GNN architectures on low-resolution image graphs built from superpixels. CNNs are highly effective for image classification, but they do not explicitly represent images as region-level structures. GNNs, on the other hand, are designed for relational data, yet they are less commonly used in beginner-level object recognition pipelines because converting images into meaningful graphs requires extra preprocessing.

This project fills that gap by building the full pipeline from image to superpixel graph to GNN classification. The work is useful because it shows not only the final accuracy numbers, but also what information is lost and preserved when an image is compressed into region-level graph nodes. In a robotics context, this representation is meaningful because robots often reason about objects, parts, surfaces, and spatial relationships rather than raw pixels alone.

3. Related Work

3.1 Graph Neural Networks

Graph neural networks generalize neural message passing to data where relationships are represented by edges. In a GNN layer, each node updates its representation by combining its own features with information from neighbouring nodes. GraphSAGE uses neighborhood aggregation and is useful for inductive graph learning. GCN uses normalized graph convolution to smooth and propagate features across adjacent nodes. GAT adds an attention mechanism so that a node can learn to weight some neighbours more strongly than others.

3.2 Superpixels for Image Representation

Superpixels reduce an image from thousands of pixels to a smaller set of meaningful regions. SLIC is a common superpixel method because it is simple, efficient, and produces compact regions that often follow visual boundaries. In this project, SLIC is used to create approximately 75 regions per image. This reduces computational complexity and creates a graph with a manageable number of nodes.

3.3 Robotic Object Recognition

Robotic object recognition requires models that can work under changing viewpoints, lighting, and object arrangements. Region-based graph representations are attractive because they can represent relationships between object parts or image regions. While this project uses CIFAR-10 rather than a physical robot dataset, it studies a perception module that could later be extended into a robotic pipeline.

4. Methodology

4.1 Dataset

The experiment uses CIFAR-10, which contains 60,000 colour images across ten object categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The original images are 32 x 32 pixels. In the supplied notebook, images are resized to 64 x 64 pixels before segmentation so that SLIC has more spatial detail to form useful regions. A subset of 2,000 training images and 500 test images is used to keep graph conversion and training practical in Google Colab.

4.2 Superpixel Segmentation

For each image, the SLIC algorithm creates approximately 75 superpixels using a compactness value of 10. SLIC groups pixels using both colour similarity and spatial closeness. The result is a segmentation map where each pixel is assigned to a region ID. Because the algorithm adapts to image content, the exact number of superpixels can vary slightly from image to image.

4.3 Graph Construction

After segmentation, each superpixel becomes a node in a graph. The feature vector for a node contains five values: mean red value, mean green value, mean blue value, normalized x-coordinate of the superpixel centroid, and normalized y-coordinate of the centroid. The graph therefore stores both visual information and rough spatial position.

Edges are created by scanning the segmentation map. If two superpixel labels touch horizontally or vertically, an edge is added between their corresponding nodes. The graph uses bidirectional edges so that information can pass in both directions during GNN message passing. This produces one graph per image, and the label of the graph is the CIFAR-10 class label.

Table 1. Image-to-Graph Construction Summary

Component	Description
Input image	CIFAR-10 RGB image resized to 64 x 64
Segmentation method	SLIC superpixels
Approximate nodes per graph	About 75 superpixels per image
Node features	Mean RGB colour and normalized centroid location
Edges	Boundary adjacency between neighbouring superpixels
Graph label	CIFAR-10 class label

4.4 Model Architectures

All three models follow the same general classification structure. They use three graph message-passing layers, nonlinear activation functions, dropout for regularization, global mean pooling to convert node embeddings into one graph-level embedding, and a final multilayer perceptron classifier. The purpose of keeping the high-level design similar is to make the comparison focus mainly on the message-passing method.

4.4.1 Baseline GNN Using GraphSAGE

The baseline model uses GraphSAGE convolution. GraphSAGE updates each node by combining its current representation with an aggregated representation of its neighbours. In this project, mean aggregation is used. This is a reasonable baseline because it is simple, inductive, and commonly used for graph learning.

4.4.2 Graph Convolutional Network

The GCN model uses normalized graph convolution. The normalization reduces the effect of nodes with many neighbours and creates a smoother propagation of information across the graph. This can be useful for superpixel graphs because most nodes represent local image regions and should exchange information with nearby regions in a balanced way.

4.4.3 Graph Attention Network

The GAT model uses attention coefficients to learn how strongly each node should attend to its neighbours. In theory, this can help the model focus on more important neighbouring regions. However, attention mechanisms also add parameters and can require more data to train reliably.

5. Experimental Setup

The supplied notebook implements the experiment in Google Colab using PyTorch, PyTorch Geometric, torchvision, scikit-image, scikit-learn, NetworkX, matplotlib, seaborn, pandas, and NumPy. The training setup uses cross-entropy loss, the Adam optimizer, a learning rate of 0.001, weight decay of 5e-4, and a step learning rate schedule. Each model is trained for 30 epochs with mini-batches of graph data.

Table 2. Experimental Configuration

Setting	Value
Training samples	2,000 graph-converted CIFAR-10 images
Test samples	500 graph-converted CIFAR-10 images
Epochs	30
Optimizer	Adam
Learning rate	0.001
Weight decay	5e-4
Batch size	32
Loss function	Cross-entropy
Pooling	Global mean pooling

6. Results

The reported results show that all three GNN models perform above the 10% random baseline for a ten-class classification problem. The GCN model achieves the best performance, followed by GAT and then the GraphSAGE baseline.

Table 3. Overall Model Performance

Model	Test Accuracy	Train Accuracy	Test Loss	Approx. Training Time
Baseline GNN / GraphSAGE	20.60%	22.30%	2.083	~420 s
GCN	29.40%	34.10%	1.882	~390 s
GAT	25.20%	26.50%	2.043	~450 s

The GCN outperforms the baseline by 8.80 percentage points and outperforms GAT by 4.20 percentage points. This suggests that normalized graph convolution is better matched to this superpixel graph representation than simple mean aggregation or attention-based aggregation under the current dataset size and feature design.

7. Per-Class Analysis

The best-performing model, GCN, performs differently across the ten CIFAR-10 categories. Classes with more distinctive colour or background patterns perform better. For example, ship has the highest F1-score because many ship images contain strong blue water or sky regions that can be captured by superpixel colour features. Horse also has high recall, likely because some horse images contain distinctive colour and shape patterns.

The weakest categories are bird and dog. These classes often require fine-grained texture or shape information, such as feathers, fur, face structure, or small object parts. Since the node features only store mean colour and centroid position, much of that fine detail is lost during superpixel aggregation.

Table 4. Per-Class Metrics for the Best GCN Model

Class	Precision	Recall	F1-Score	Support
Airplane	0.35	0.39	0.37	46
Automobile	0.28	0.31	0.29	52
Bird	0.12	0.02	0.03	51
Cat	0.18	0.18	0.18	50
Deer	0.28	0.25	0.26	53
Dog	0.20	0.06	0.09	53
Frog	0.29	0.40	0.34	53
Horse	0.25	0.51	0.33	35
Ship	0.37	0.52	0.43	60
Truck	0.37	0.36	0.37	47

8. Discussion

8.1 Why GCN Performed Best

The GCN model likely performs best because its normalized aggregation is well suited to the local structure of superpixel graphs. Superpixel graphs are relatively regular, but the number of neighbours can still vary by region. Normalization helps prevent high-degree nodes from dominating the representation. This is important when large uniform image regions touch many smaller regions.

8.2 Why GAT Did Not Beat GCN

GAT is more flexible because it learns attention weights between neighbouring nodes. However, flexibility can also make training harder when the dataset is small and node features are simple. With only five node features and 2,000 training graphs, the attention mechanism may not receive enough information to learn stable neighbour importance patterns. This explains why GAT improves over the baseline but does not surpass GCN.

8.3 Why the Absolute Accuracy Is Limited

The final accuracies are low compared with modern CNN results on CIFAR-10, but this is expected. The graph representation uses only mean colour and spatial position for each superpixel. It does not include texture descriptors, deep visual embeddings, edges, gradients, or shape descriptors. CIFAR-10 images are also low-resolution, which makes it difficult for SLIC to produce semantically clean object parts. The small training subset further limits generalization.

8.4 Relevance to Robotics

Even with limited accuracy, the project is valuable for robotics because it demonstrates a compact region-based perception representation. A robot may benefit from reasoning over object parts, surfaces, or segmented regions rather than raw pixels alone. Superpixel graphs can also be extended with depth, geometry, tactile information, or language labels in future robotic systems.

9. Limitations

- The experiment uses a subset of CIFAR-10 rather than the full dataset.
- Node features are limited to mean RGB colour and normalized position.
- No edge features are used, even though boundary strength and distance could improve graph reasoning.
- No CNN or Vision Transformer baseline is included for direct comparison with standard image models.
- Only one main SLIC configuration is tested, so the effect of different superpixel counts is not fully explored.
- The dataset is not a real household robotics dataset, so robotics conclusions are conceptual rather than deployed on a physical robot.

10. Future Work

- Train on the full CIFAR-10 dataset and evaluate multiple random seeds for stronger statistical reliability.
- Add richer node features such as texture descriptors, gradients, local binary patterns, or pretrained CNN embeddings.
- Add edge features such as boundary contrast, centroid distance, and relative direction.
- Test different superpixel counts, such as 25, 50, 100, and 150 regions per image.
- Compare against CNN and Vision Transformer baselines to measure the trade-off between grid-based and graph-based perception.
- Use graph pooling methods such as DiffPool, TopKPooling, or SAGPooling to learn hierarchical region representations.
- Apply the method to a robotic object dataset with household items, RGB-D data, or real camera images.
- Integrate the perception model into a visual-language-action pipeline where graph-level object recognition supports action planning.

11. Conclusion

This project presented a complete graph-based image classification pipeline using SLIC superpixels and graph neural networks. Each CIFAR-10 image was converted into a graph where superpixels became nodes and spatially adjacent regions became edges. Three GNN models were compared: a GraphSAGE baseline, GCN, and GAT. The GCN model achieved the best test accuracy of 29.40%, showing that normalized graph convolution is effective for this type of compact superpixel representation.

The results also show the limitations of simple graph features. Mean colour and position are not enough to fully capture fine-grained object differences, especially for visually similar animal categories. However, the project successfully demonstrates the core idea that images can be represented as graphs and classified using GNNs. With richer features, larger datasets, and stronger graph pooling methods, superpixel GNNs could become a useful component in robotic perception systems.

Statement of Contribution

The supplied project materials identify Harmit Patel as the author of the original notebook and draft report. The experimental pipeline, code, graph construction method, model training, and reported results are based on those supplied files. This polished version reorganizes and expands the explanation into a clearer research-paper format.

AI Usage Statement

AI assistance was used to organize the explanation, improve grammar and formatting, and prepare a polished README and research-paper version based on the supplied project files. The student should review, verify, and understand all technical content before submission.

References

[1] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Susstrunk, "SLIC Superpixels Compared to State-of-the-Art Superpixel Methods," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 34, no. 11, pp. 2274-2282, 2012.

[2] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," Advances in Neural Information Processing Systems, 2017.

[3] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," International Conference on Learning Representations, 2017.

[4] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph Attention Networks," International Conference on Learning Representations, 2018.

[5] A. Krizhevsky and G. Hinton, "Learning Multiple Layers of Features from Tiny Images," University of Toronto Technical Report, 2009.

[6] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The Graph Neural Network Model," IEEE Transactions on Neural Networks, vol. 20, no. 1, pp. 61-80, 2009.

[7] F. Monti, D. Boscaini, J. Masci, E. Rodola, J. Svoboda, and M. M. Bronstein, "Geometric Deep Learning on Graphs and Manifolds Using Mixture Model CNNs," IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2017.

[8] B. Knyazev, G. W. Taylor, and M. R. Amer, "Understanding Attention and Generalization in Graph Neural Networks," Advances in Neural Information Processing Systems, 2019.

[9] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical Graph Representation Learning with Differentiable Pooling," Advances in Neural Information Processing Systems, 2018.

[10] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," International Conference on Learning Representations, 2015.

Appendix A: Notebook Execution Summary

1. Check the PyTorch runtime and CUDA availability.
2. Install PyTorch Geometric and related graph-learning dependencies.
3. Restart the runtime after installation.
4. Import PyTorch, PyTorch Geometric, scikit-image, NetworkX, matplotlib, sklearn, NumPy, and pandas.
5. Download CIFAR-10 using torchvision.
6. Visualize one sample image per class.
7. Define the SLIC-based image-to-graph conversion function.
8. Generate superpixel visualizations and graph examples.
9. Convert the selected training and test images into PyTorch Geometric Data objects.
10. Define GraphSAGE, GCN, and GAT model classes.
11. Train all three models and record accuracy/loss curves.
12. Generate result plots, confusion matrix, classification report, and saved model files.